

Shadows of Neon City

Write-Up by @PHPCore

Description

I cracked the shard's stego layers and extracted the relevant data. Thought it'd be a piece of cake, but it's a bit more involved. We don't have a direct private key, just a public key and the encrypted payload. The suits at AstraCorp might have gotten lazy—they used a small RSA modulus.

Research

When reading the last sentence of the description, I immediately thought about cracking a weak RSA-Key. The [recommended key-length by ENISA](#) is at least 1024-Bits for legacy systems and 15360-Bits for long-term security. When we look into the attachment, that is provided with the challenge, we see the target application, written in Python, called *app.py*. A look into the source-code reveals:

```
import random
from sympy import nextprime

def gen_prime(bits):
    candidate = random.getrandbits(bits)
    if candidate % 2 == 0:
        candidate += 1
    return nextprime(candidate)

def generate_rsa_keypair():
    e = 65537
    bits = 64

    p = gen_prime(bits)
    q = gen_prime(bits)
    while q == p:
        q = gen_prime(bits)

    n = p * q
    phi = (p - 1) * (q - 1)
    d = pow(e, -1, phi)

    return n, e, d, p, q
```

They indeed use RSA and the used RSA-Key generator creates two random prime numbers with a length of 64-Bit. This will result in a key-length of 128-Bit. This is far below the recommended size. So we can assume, that we can easily compromise this encryption, by factoring the public modulus of the RSA-Key *n*, which is the product of both prime numbers. So we just need to look at the rest of the challenge, to understand, how the challenge works:

```

chall_text, pwd = gen_challenge()
client_socket.sendall(chall_text.encode())

client_socket.sendall(b"\nEnter password:\n")
answer = client_socket.recv(4096).decode().strip()

if answer == pwd:
    msg = f"V, inject this code into the mainframe backdoor: {flag}\n"
else:
    if wrong_pw_count > 10:
        msg = "Shard locked\n"
    else:
        msg = "Invalid access code\n"
        wrong_pw_count += 1

client_socket.sendall(msg.encode())

```

This is a snippet of the main-function from the Python script, which I stripped down to the essential part of this function: How the user interacts with the system. It first generates a challenge using a call to `gen_challenge`. Then it prints out the challenge-text to the user and expects the user, to enter the generated password to get to the flag. Lets look into `gen_challenge` to understand, the given text and password.

```

import base64
import os

def gen_challenge():
    access_pwd = os.urandom(6).hex()
    n, e, d, p, q = generate_rsa_keypair()

    m_int = int.from_bytes(access_pwd.encode(), byteorder="big")
    c_int = pow(m_int, e, n)
    c_bytes = c_int.to_bytes((c_int.bit_length() + 7) // 8, "big")
    c_b64 = base64.b64encode(c_bytes).decode()

    chall_text = f"----- SECRET MESSAGE FROM NEON CITY MAYOR ----- \n" \
        "n = {n} \n" \
        "e = {e} \n" \
        "cipher = {c_b64} \n" \
        "----- \n"
    return chall_text, access_pwd

```

For the sake of readability, I modified the `chall_text` variable a bit. If we look into this function, it generates 6-Bytes of secure randomness and sets the expected password to the hexadecimal representation of those bytes. Then it generates a RSA-Keypair using the flawed RSA-Key generator. The function then proceeds to encrypt the password using the public-key and encodes the result in Base64. The challenge-text is a combination of the encrypted password and the RSA public-key.

So we receive the public-key of a weakly generated RSA-Keypair, which means, that we are able compromise the RSA-Keypair by prime-factorizing the modulus to get our initial prime numbers. With the private-key we can then decrypt the password.

Solution

We first start with compromising the private-key. For this we take the public key and proceed to factor the modulus, which results in the original prime numbers. Then we calculate our private-key using our now known prime numbers and pass them into a PrivateKey (In this code `n_line` and `e_line` are the output lines by the challenge as Python `bytes` objects):

```
import rsa
from sympy.ntheory import factorint

n = int(n_line.split(b'= ')[-1].decode())
e = int(e_line.split(b'= ')[-1].decode())

primes = factorint(n)
p, q = primes.keys()

phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)

pkey = rsa.PrivateKey(n=n, e=e, d=d, p=p, q=q)
```

Now that we have the private-key, we now just need to decrypt the password, similarly to how it was encrypted, using the following snippet (`cipher_lines` is also the output from the challenge as a `bytes` object):

```
cipher = cipher_line.split(b'= ')[-1].decode()
cipher_int = int.from_bytes(base64.b64decode(cipher), 'big')
decrypt_int = pkey.blinded_decrypt(cipher_int)
pwd = decrypt_int.to_bytes((decrypt_int.bit_length() + 7) // 8,
byteorder="big").decode()
print(pwd)
```

When we then enter the now decrypted password into the challenge, we are greeted with the following message:

```
V, inject this code into the mainframe backdoor:
dach2025{curs3d_RSA_1s_curs3d_2124214}
```

And now we have the flag, which we can enter into the CTF-Platform.

Patching the Vulnerability

This system is flawed in multiple ways. First we will fix the obvious and exploitable vulnerability by choosing a larger key-size, like the one recommended by ENISA (15360-Bit recommended or at least 1024-Bit). Other flaws which aren't relevant to this challenge, but are relevant if you use such crypto-systems, exists in this challenge. Like the try to implement well known cryptography by yourself, which will almost every time result in vulnerabilities, which is why you should use established libraries like PyCryptodome or OpenSSL. Another flaw would be encrypting your message with RSA directly, instead of switching to a symmetric Encryption and sharing the symmetric-key by encrypting it using RSA. And what should also be done, when we look at the context: It tells us in the challenge message: "A secret message from the mayor", when we assume they really want to exchange messages in a secret way, there is one essential part missing in this message exchange: A way to verify the authenticity of the message like HMAC-SHA256, to verify that the message wasn't tampered with on the way.